

ReadQuest

Plano Técnico de Produto — Leitor de PDF como Core

Documento gerado para leitura do Claude Code. Consolida a definição de escopo, decisões de arquitetura, sistema de design de referência (Fable) e roadmap de execução após o pivot: o coração do produto é um leitor de PDF que substitui o Kindle. Tracker, notas, dashboard e IA são agregadores de valor, construídos depois do reader estar sólido.

Gerado em 09/07/2026 · Persona: PO/Tech Lead técnico

0. Definição do Core do Produto

O ReadQuest é, antes de tudo, um **app-website de leitura de PDF** que substitui o Kindle para livros em PDF no PC. O usuário carrega seu PDF e tem uma experiência de leitura superior a abrir o arquivo em um leitor genérico (SumatraPDF, Edge, Xodo, Adobe Reader).

Tudo o que não é o reader em si — biblioteca, sessões, dashboard, notas, review, exportação, IA — é tratado como **camada de valor agregado**, construída em cima do core, nunca antes dele.

1. Análise de Viabilidade Técnica e Riscos

Mudança de prioridade

- PDF.js deixa de ser o item isolado de maior risco no fim do roadmap (antiga Sprint 7) e passa a ser o bloqueador do MVP inteiro.
- É obrigatório um **spike técnico (Sprint 0)** antes de comprometer prazo de qualquer sprint de produto.
- Tracker, notas e dashboard passam a ser "agregadores de valor" — vêm depois do reader validado.

Sobre o design de referência (Fable)

- O **DESIGN_-Fable.md** é um design system editorial de marketing/landing page: bandas full-bleed coloridas (Fable Forest, Storybook Sky), ilustrações grandes, headlines 48–80px. Ótimo para a home/onboarding do app — não aplicável diretamente à tela de leitura.
- Os tokens **Paper Cream (#f7f4ee)** + **Ink (#161015)** são, na prática, a paleta de um livro físico / Kindle — base natural para o **modo de leitura claro (Paper)**, mais indicado que dark mode como padrão para leitura longa de texto denso.
- Decisão: **Paper** como tema padrão da Reading Surface (equivalente ao papel do Kindle), com **Dark** como tema alternativo para leitura noturna. O App Shell (biblioteca, dashboard) pode manter dark mode como padrão, se preferido.
- Claude Code fica livre para compor o layout usando os tokens (cor, tipografia, radius, spacing) como linguagem de design — sem replicar literalmente os componentes de landing (bandas, ilustrações) dentro do reader.

Riscos ordenados por probabilidade × impacto

Risco	Prob.	Impacto	Nível
PDF.js não performar bem com PDFs grandes/escaneados sendo o core	Alta	Crítico	■ Alto
Aplicar o design de landing (bandas, ilustrações) dentro da tela de leitura	Média	Alto	■ Alto

Fonte Heldane Display não licenciada / precisa de substituto	Alta	Baixo	■ Baixo
Texto selecionável para highlights (feature futura) no PDF.js	Média	Médio	■ Médio
Fricção no cadastro se upload de PDF virar obrigatório	Baixa	Médio	■ Médio

Mitigações

- Spike de PDF.js antes de qualquer UI — testar **pdfjs-dist** com PDF leve, PDF pesado (com imagens) e PDF escaneado, medindo tempo de render.
- Separar dois modos visuais explícitos: **Reading Surface** (minimalista, paper cream, foco total no texto) vs **App Shell** (biblioteca, dashboard, onboarding — aqui cabe a linguagem editorial completa do Fable).
- Substituir **Heldane Display** por **Fraunces** (Google Fonts, open-source, sugerido no próprio design doc).
- Cadastro de livro passa a exigir upload de PDF — fileRef deixa de ser opcional no modelo de dados.

Decisões de arquitetura confirmadas

- Renderização **página-única** (não scroll contínuo) — mais leve, mais fiel à metáfora de livro/Kindle.
- Capa do livro: **as duas opções disponíveis** — extração automática da primeira página do PDF como thumbnail, e upload manual alternativo. O usuário escolhe qual usar por livro.
- PDF armazenado como Blob no **IndexedDB** (não LocalStorage).
- **pdfjs-dist** rodando em Web Worker, para não travar a UI durante o render.
- Progresso de leitura salvo automaticamente por bookId (número da página atual).
- Dois temas de leitura no mínimo: **Paper** (padrão) e **Dark**, desacoplados do tema do App Shell.

2. Arquitetura de Dados e Fluxo

Fluxo principal

```
[Upload PDF] -> Blob armazenado no IndexedDB (bookId como chave)
|
[pdfjs-dist worker] -> extrai metadata (n. paginas) -> auto-preenche totalPages
|
[Reading Surface] -> renderiza pagina unica, controla zoom/navegacao
|
a cada mudanca de pagina -> debounce -> updateProgress(bookId, currentPage)
|
[App Shell / Biblioteca] <- reflete progresso atualizado nos BookCards
```

Camadas visuais (separação explícita)

App Shell (tokens Fable completos)

- Landing/Onboarding -> bandas full-bleed, ilustracao, Fraunces display
- Biblioteca -> Cream Card, Book Cover Tile, grid de capas
- Dashboard/Notas -> Cream Card, pill buttons, tipografia Inter

Reading Surface (subset minimalista dos tokens)

- Fundo: --color-paper-cream (padrao) ou tema dark
- Texto/chrome: --color-ink
- Sem bandas coloridas, sem ilustracoes, sem headline display
- Controles de navegacao flutuantes, discretos, somem ao ler

Modelo de dados — Book (atualizado)

```
type Book = {
  id: string;
  title: string;
  author?: string;
  totalPages: number;          // auto-preenchido pelo pdf.js
  currentPage: number;
  status: 'want_to_read' | 'reading' | 'paused'
        | 'completed' | 'abandoned';
  category: string;
  tags: string[];
  coverSource: 'extracted' | 'manual'; // escolha do usuario
  coverUrl?: string;                // extraida do PDF ou upload manual
  fileRef: string;                  // chave do Blob no IndexedDB (obrigatorio)
  readingTheme?: 'paper' | 'dark';  // preferencia por livro
  startedAt?: string;
  completedAt?: string;
  createdAt: string;
  updatedAt: string;
};
```

Modelo de dados — ReadingSession e Note

```
type ReadingSession = {
  id: string;
  bookId: string;
  date: string;
  startPage: number;
  endPage: number;
  durationMinutes: number;
  notes?: string;
  mood?: 'focused' | 'tired' | 'curious' | 'distracted' | 'immersed';
  sessionType?: 'leisure' | 'study' | 'review' | 'english' | 'research';
  createdAt: string;
```

```
};

type Note = {
  id: string;
  bookId: string;
  sessionId?: string;
  page?: number;
  title?: string;
  content: string;
  type: 'idea' | 'quote' | 'concept' | 'character' | 'scene'
    | 'reflection' | 'question' | 'review' | 'vocabulary' | 'reference';
  tags: string[];
  favorite: boolean;
  createdAt: string;
  updatedAt: string;
};
```

3. Roadmap do Projeto

Sprint 0 — Spike Técnico PDF.js

Duração estimada: 2-3 dias

Objetivo: Validar viabilidade de performance do pdfjs-dist antes de comprometer prazo

Critério de Aceite: PDF de 300+ páginas renderiza página única em <500ms, sem travar a UI

Entregas:

- Protótipo isolado com pdfjs-dist + Web Worker
- Teste com PDF leve, PDF pesado (com imagens) e PDF escaneado
- Confirmar renderização página-única (decisão já tomada)

Sprint 1 — Leitor PDF Core

Duração estimada: 1.5–2 semanas

Objetivo: Abrir e ler um PDF localmente, substituindo o Kindle no essencial

Critério de Aceite: Usuário sobe um PDF, lê com navegação/zoom, fecha e volta exatamente onde parou

Entregas:

- Setup React + Vite + TS + Tailwind + tokens Fable (Paper Cream/Ink)
- Upload local de PDF -> Blob no IndexedDB
- Renderização página-única com pdfjs-dist
- Navegação por página (teclado + clique) e zoom
- Salvar página atual automaticamente
- Modo foco (esconde chrome, só o texto)

Sprint 2 — Refino da Reading Surface

Duração estimada: 1 semana

Objetivo: Experiência de leitura no nível "melhor que Kindle"

Critério de Aceite: Leitura confortável em sessões longas, temas Paper e Dark funcionais

Entregas:

- Tema Paper (padrão) e Dark aplicando Ink/Paper Cream corretamente
- Atalhos de teclado (página anterior/próxima, zoom, fullscreen)
- Transição suave entre páginas (Framer Motion, discreta)
- Indicador de progresso (nº página / total) sutil

Sprint 3 — Biblioteca (App Shell)

Duração estimada: 1 semana

Objetivo: Gerenciar múltiplos livros carregados

Critério de Aceite: Usuário vê todos os PDFs carregados, com capa e progresso, e abre qualquer um

Entregas:

- CRUD de metadados do livro (título, autor, categoria, status, tags)
- Capa: extração automática da 1ª página OU upload manual — usuário escolhe
- Grid de biblioteca com Book Cover Tile (estilo Fable)
- Status e categorias como filtro

Sprint 4 — Sessões de Leitura & Progresso

Duração estimada: 1 semana

Objetivo: Tracking automático de progresso ao longo do tempo

Critério de Aceite: Sessão é registrada automaticamente ao ler, sem ação manual obrigatória

Entregas:

- Captura automática de sessão (tempo + páginas) ao navegar no reader
- Edição manual opcional (ajustar tempo, adicionar observação)
- Histórico de sessões por livro

Sprint 5 — Dashboard

Duração estimada: 3-5 dias

Objetivo: Visão consolidada da atividade de leitura

Critério de Aceite: Usuário entende sua atividade da semana em poucos segundos

Entregas:

- Selectors derivados (páginas/semana, tempo total, streak)
- Cards de livros em andamento/concluídos
- Última sessão + média de páginas/sessão

Sprint 6 — Notas

Duração estimada: 1 semana

Objetivo: Anotações vinculadas à página lida

Critério de Aceite: Usuário cria nota durante a leitura, vinculada à página atual

Entregas:

- Painel lateral de notas dentro do próprio Reading Surface
- CRUD de nota com tipo/tag
- Lista de notas por livro fora do reader

Sprint 7 — Export Markdown

Duração estimada: 3-5 dias

Objetivo: Tornar os dados portáteis

Critério de Aceite: Usuário exporta .md de sessão/notas/livro e usa no Obsidian

Entregas:

- Funções puras de serialização MD (livro, sessão, notas)
- Copiar para clipboard
- Baixar arquivo .md

Sprint 8 — Modo Review

Duração estimada: 1 semana

Objetivo: Apoiar a produção de reviews ao concluir um livro

Critério de Aceite: Usuário termina o livro com base pronta para escrever review

Entregas:

- Schema Review + CRUD
- Formulário estruturado (pontos fortes/fracos, temas, cenas, nota final)
- Export da review em Markdown

Sprint 9 — IA Assistida (fase 1: sem API)

Duração estimada: 1 semana

Objetivo: Transformar notas em material de estudo/review via prompt exportável

Critério de Aceite: App gera prompt copiável com contexto de notas/sessão para colar em Claude/ChatGPT

Entregas:

- Templates de prompt (resumo, perguntas de revisão, base de review)
- Botão "copiar prompt" com dados interpolados
- (Futuro/opcional) integração direta via API

4. Backlog Detalhado — Sprint 0 + Sprint 1

[Spike] Validação PDF.js (fazer antes de tudo)

- [Config] Instalar pdfjs-dist e configurar worker (pdf.worker.min.js) via Vite
- [Teste] Renderizar um PDF pequeno (<20 páginas) em <canvas>, medir tempo de primeira renderização
- [Teste] Renderizar um PDF grande (300+ páginas, com imagens), medir tempo de navegação entre páginas
- [Teste] Testar um PDF escaneado — confirmar exibição visual mesmo sem texto selecionável nessa fase
- [Config] Confirmar implementação de renderização página-única (decisão já tomada)

[Setup] Projeto Base

- [Config] npm create vite@latest readquest -- --template react-ts
- [Config] Tailwind + tokens do Fable como CSS custom properties (--color-paper-cream, --color-ink, etc.)
- [Config] Fonte: Fraunces (Google Fonts) no lugar de Heldane Display
- [Config] Instalar zustand, idb-keyval, pdfjs-dist, framer-motion, lucide-react

[Estado] Persistência de PDF

- [Estado] lib/db.ts — salvar/recuperar Blob de PDF por bookId no IndexedDB
- [Estado] stores/useBookStore.ts com addBook (recebe File, extrai metadata via pdf.js, salva Blob)
- [Teste] Confirmar que o PDF persiste após reload (F5) sem precisar re-upload

[Frontend] Upload e Reading Surface

- [Frontend] Componente UploadDropzone (drag-and-drop + seleção manual de arquivo)
- [Frontend] Componente PdfReader — renderiza página atual via canvas do pdf.js
- [Frontend] Controles de navegação (botão anterior/próxima + atalho de teclado ←/→)
- [Frontend] Controle de zoom (+/-, reset)
- [Frontend] Modo foco — esconde header/chrome, mostra só o canvas + controles mínimos flutuantes
- [Frontend] Escolha de capa: opção "usar primeira página" (auto) ou "enviar imagem" (manual) no cadastro
- [UX] Aplicar --color-paper-cream como fundo da Reading Surface, --color-ink para textos de UI

[Estado] Progresso Automático

- [Estado] Ao trocar de página, debounce de 1s -> updateBook(bookId, { currentPage })
- [Estado] Ao reabrir um livro já iniciado, reader pula automaticamente para a currentPage salva

[Teste] Critérios de Aceite Manuais

- [Teste] Upload de PDF de 300+ páginas, navegar até página 150, fechar e reabrir — confirma retorno na página 150
- [Teste] Zoom in/out mantém a página atual visível corretamente
- [Teste] Modo foco esconde toda a UI exceto texto e controles mínimos
- [Teste] Cadastrar um livro escolhendo capa manual e outro com capa extraída automaticamente — ambos exibem corretamente na biblioteca